

Simulation of Reach Alpha5 Manipulator using ROS2

Tony Tang

June 23rd – August 14th, 2025

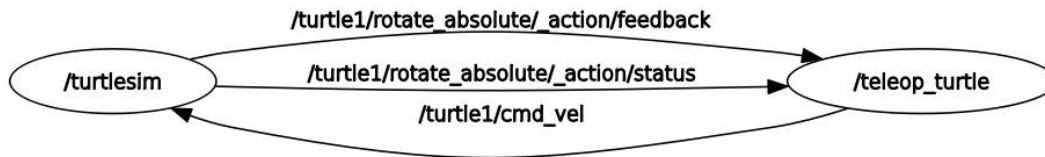
Introduction

Underwater vehicles have been widely adopted by aquacultural industries to perform repairs and maintenance, and thus minimizing the need for human interventions. By deploying such vehicles, one can significantly decrease costs and reduce the risk of injuries to human divers.

At SINTEF Ocean, I worked on integrating the Reach Alpha 5 manipulator with the BlueROV2 so that it can navigate the salmon farms while repairing any damage on cage nets. The BlueROV2 has 4 DOF and the Reach Alpha 5 manipulator also has 4 DOF, which gives the integrated system a total of 8 DOF (including roll and pitch, it can have 2 more DOF for a total of 10 DOF). This enables the robotic arm to reach any holes detected by the vehicle. My project is to create a simulation of the Reach Alpha 5 moving to desired Cartesian coordinates. The simulation will allow researchers at SINTEF to better understand how the robotic arm moves before physically using it, ensuring they stay within operational limits to prevent damage. I plan to use ROS2 because it utilizes Multi-Threaded Executor to run multiple callbacks in parallel rather than one at a time.

Understanding ROS2

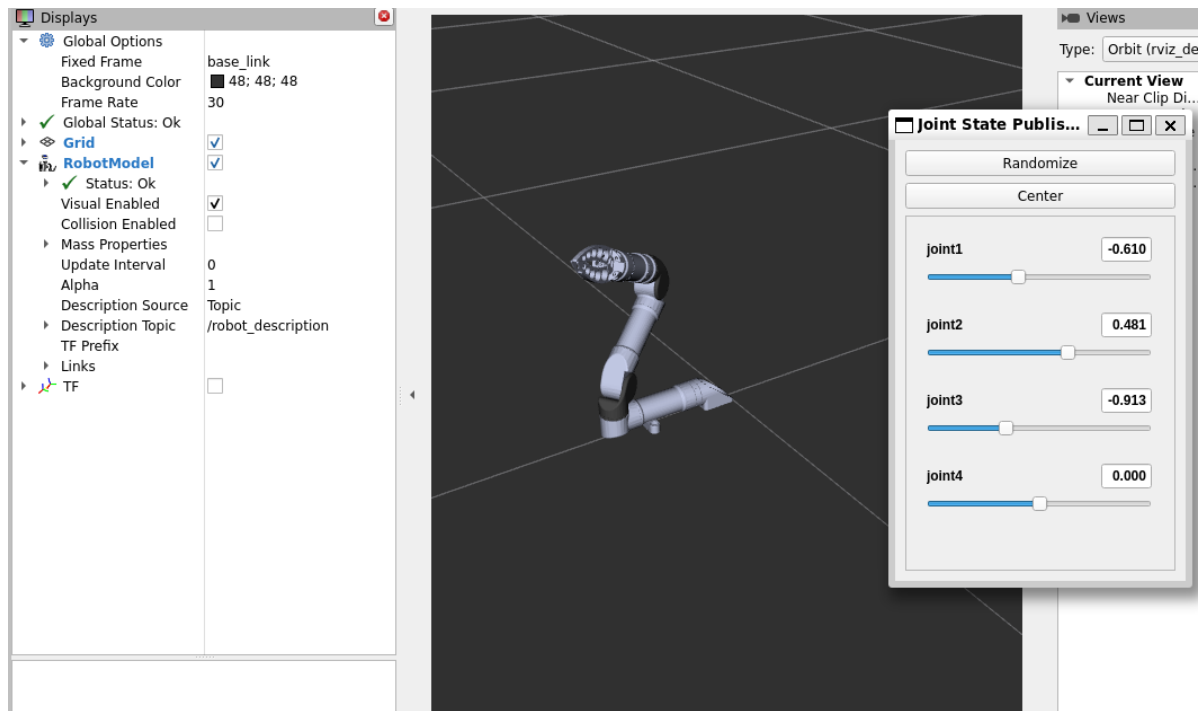
Before starting the internship, I had no prior experiences with ROS2, so I dedicated this summer to building the simulation by following tutorials. I installed Ubuntu (22.04) and ROS2 Humble and looked through the documentations at [Humble Tutorials](#). The first section introduced nodes, publishers, subscribers, and topics using a package named turtlesim. Below is a `rqt_graph` – a graphical tool for visualizing how nodes in a system are connected – illustrating two nodes from turtlesim communicating with each other:



In ROS2, nodes are the most fundamental units, each responsible for a specific task, such as `/turtlesim` and `/teleop_turtle` in this case. Nodes can be both publishers and subscribers, meaning that they can send or receive messages via topics, respectively. Topics are communication channels between nodes, and in this example, they are: `/turtle1/rotate_absolute/_action/feedback`, `/turtle1/rotate_absolute/_action/status`, and `/turtle1/cmd_vel`. The arrows represent the direction of message flowing from one node to another over a topic. For example, an arrow points from `/teleop_turtle` to `/turtlesim` over `/turtle1/cmd_vel`, so `/teleop_turtle` is a publisher and `/turtlesim` is a subscriber. The other two arrows point in the opposite direction from `/turtlesim` to `/teleop_turtle`, indicating that `/turtlesim` is a publisher and `/teleop_turtle` is a subscriber. This demonstrates that a node can be both a publisher and subscriber, as previously established.

RViz

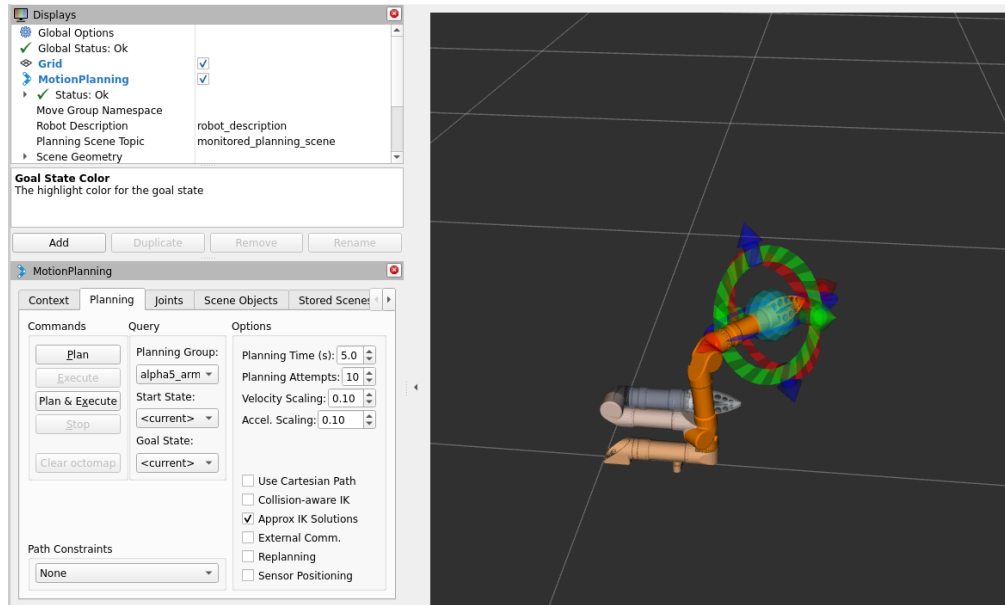
After understanding the basic concepts in ROS2, I began working with RViz to develop a visualization of the Reach Alpha 5. RViz is a 3D visualization tool for robots and is useful in my case because it illustrates the joint states. I used the mesh files for the robotic arm and converted it from STL to URDF via SolidWorks, which enables me to see how the arm rotates.



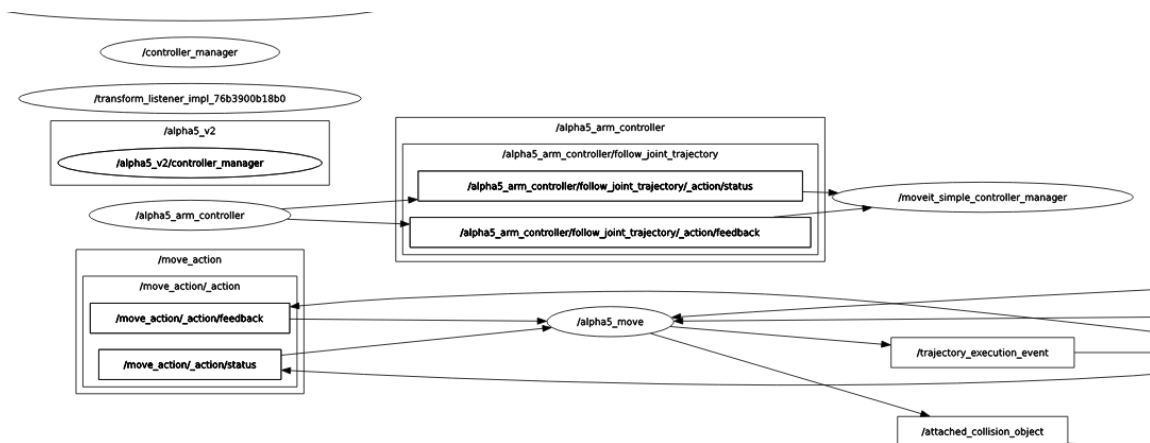
The image above illustrates the Reach Alpha 5 manipulator at its current joint states. Joint State Publisher is a node that enables the user to independently rotate each joint, which moves the position of the end effector via forward kinematics. There are four sliders, each representing a degree of freedom of the arm.

Movelt: Motion Planning

The main goal of the simulation is to visualize how the Reach Alpha 5 manipulator moves given a Cartesian coordinate. This requires inverse kinematics, where the input is the XYZ coordinates and outputs are the corresponding joint states – opposite of controlling the joint state publisher. I planned to integrate Movelt into RViz, as it conveniently has a built-in inverse kinematics solver that converts a position into joint states. Being unfamiliar with Movelt, I followed its [documentation](#) and [Setup Assistant](#) guides to configure the robotic arm.



The figure above illustrates the Reach Alpha 5 manipulator after dragging the end effector to a certain position. Since the robotic arm only has 4 DOF, an approximate inverse kinematics solution was most likely used to compute the joint states. The final step for the simulation was to create a node called Alpha5_move, which takes in a Cartesian coordinate and moves the robot arm to that position. This was the most challenging step as following the tutorials yielded no visible results or error messages, indicating missing nodes and/or unconnected nodes. Ultimately, I was able to develop a working Alpha5_move node by comparing the rqt_graph of my package with the tutorials'. I discovered that I was missing the alpha_arm_controller, the node that enables the arm to move functionally. After making necessary edits to Alpha5_move and the launch code, the arm moved to the input position. A demonstration of how to use the packages is shown below along with a separate video.



Demonstration

1. Go to the workspace and run `ros2 launch alpha5_arm_config_v3 demo.launch.py`

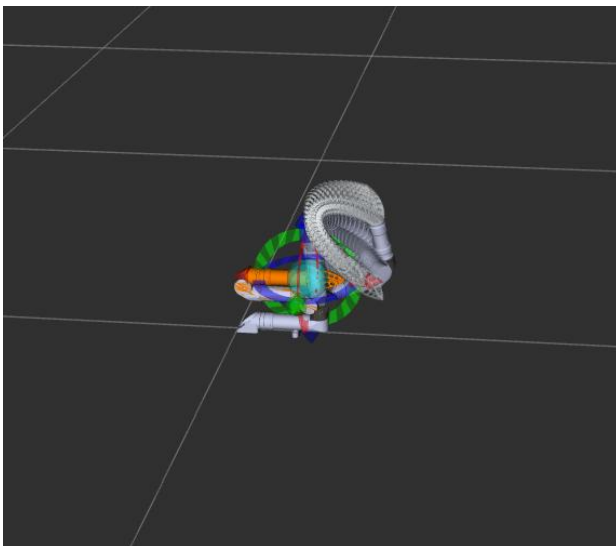
```
tonyt114@Tony:~/ws_moveit$ ros2 launch alpha5_arm_config_v3 demo.launch.py
```

2. Uncomment the following lines with the desired XYZ values and paste them into a separate terminal. You should see “set parameter successful”.

```
// uncomment and copy and paste into terminal with the values  
// ros2 param set /alpha5_move target_x 0.3  
// ros2 param set /alpha5_move target_y 0.11  
// ros2 param set /alpha5_move target_z 0.2
```

```
tonyt114@Tony:~$ ros2 param set /alpha5_move target_x 0.3  
ros2 param set /alpha5_move target_y 0.11  
ros2 param set /alpha5_move target_z 0.2  
Set parameter successful  
Set parameter successful  
Set parameter successful
```

3. The arm moves to that position!



Additional Notes:

Workspace organization:

